

Remote PLC Diagnostics Over Encrypted Mesh Networks

Secure Industrial Automation via Tailnet-Bridged Modbus TCP

Version 1.0 | February 18, 2026 Authors: Human-AI Collaborative Development (FactoryLM Team)

Abstract

This paper presents a novel approach to remote industrial PLC diagnostics that eliminates the traditional trade-off between accessibility and security. By bridging Modbus TCP communication through an encrypted mesh network (WireGuard-based VPN), we demonstrate real-time read/write access to an Allen-Bradley Micro 820 PLC from a cloud server with sub-35ms round-trip latency — without exposing any industrial control ports to the public internet.

The system was verified on February 18, 2026 with live hardware: coil states were read, program variables were written and confirmed, and physical I/O panel states were observed in real-time from a remote server traversing two network hops.

Key result: A cloud-hosted diagnostic service can securely monitor and interact with an air-gapped PLC at network latencies comparable to local HMI response times, using only commodity hardware and open-source software.

PART 1: THE SCIENCE

1. Problem Statement

Industrial PLCs (Programmable Logic Controllers) are the backbone of factory automation. They control motors, valves, conveyors, and safety systems. Monitoring their state — reading sensor values, checking fault codes, verifying I/O — is essential for maintenance and diagnostics.

The problem: PLCs sit on isolated industrial networks (typically 192.168.x.x or 10.x.x.x subnets) with no internet access. This is intentional — connecting a PLC to the internet is a serious security risk. But it also means:

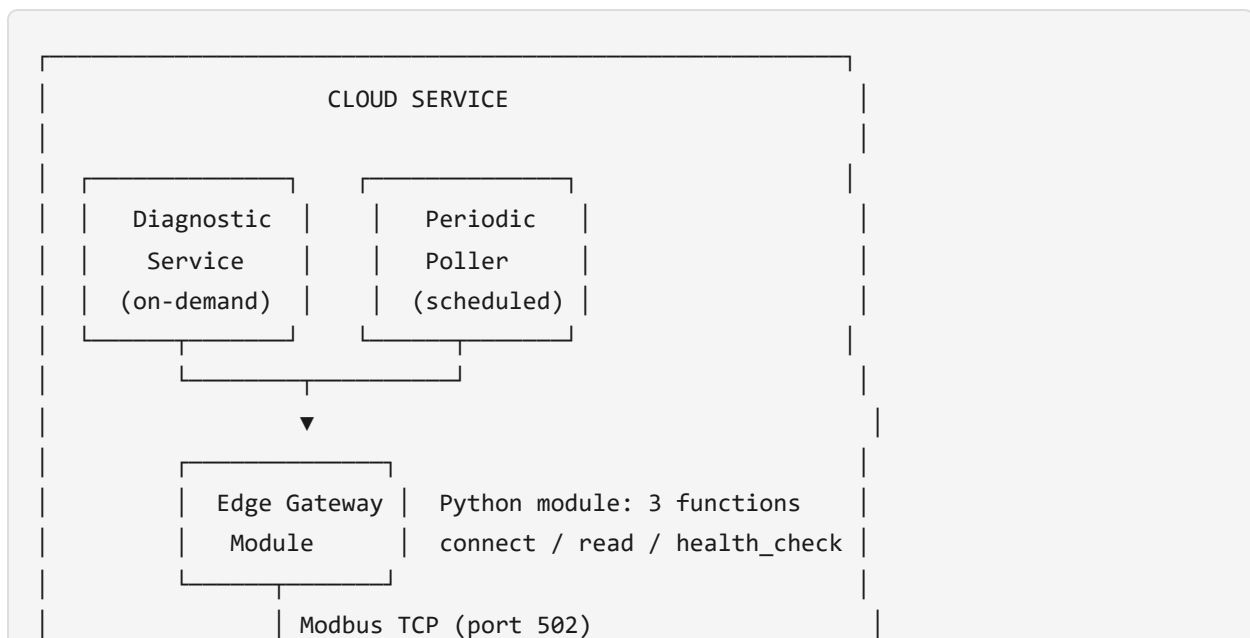
- Technicians must be physically present to diagnose issues
- Remote experts cannot assist without expensive site visits
- Predictive maintenance requires manual data collection
- After-hours alarms go uninvestigated until morning

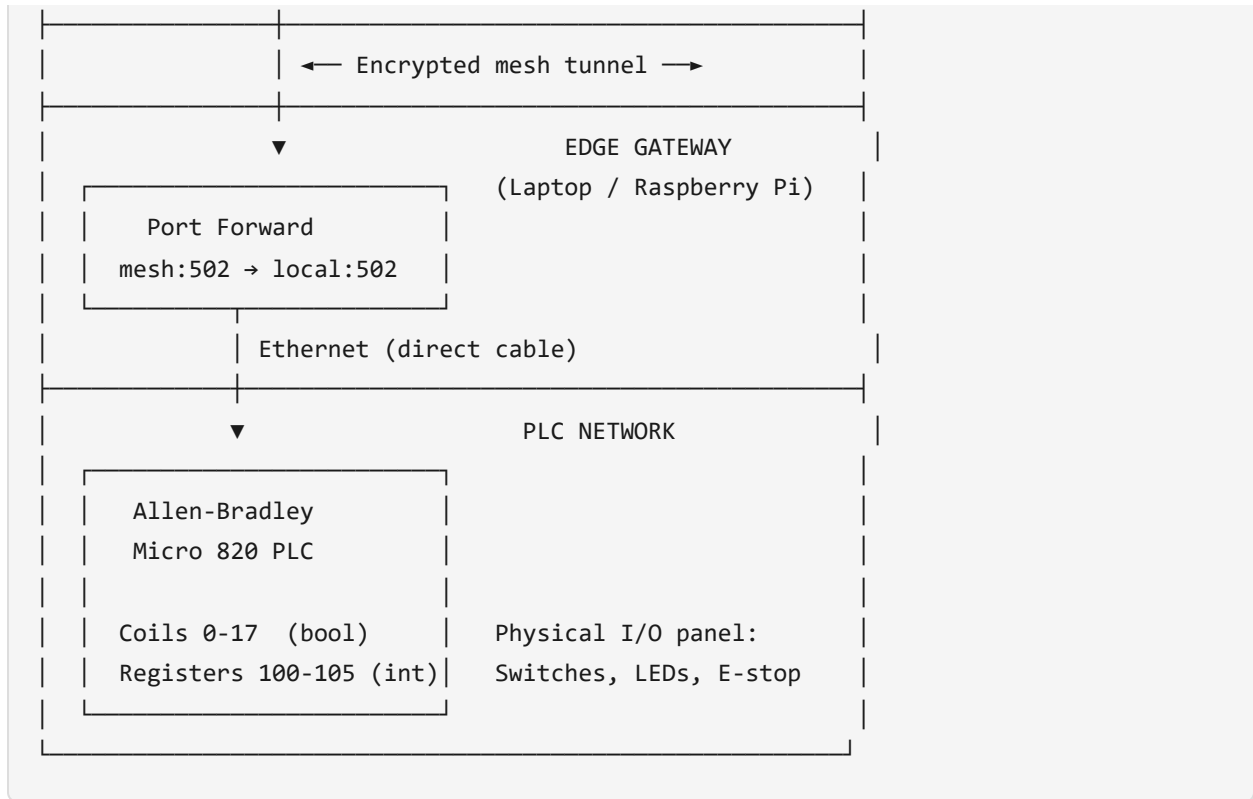
Traditional solutions (industrial VPNs, DMZ architectures, cloud SCADA gateways) are expensive (\$5,000–\$50,000+), complex to configure, and introduce attack surface. They require IT/OT convergence expertise that most small-to-medium manufacturers lack.

Our approach: Use an encrypted peer-to-peer mesh network to bridge a standard laptop (acting as edge gateway) between the PLC's local network and a cloud diagnostic service. No public ports. No firewall rules. No IT department required.

2. Architecture

The system consists of three nodes connected via an encrypted mesh overlay network:





Protocol Stack:

Layer	Cloud → Edge	Edge → PLC
Application	JSON/HTTP REST API	Modbus TCP
Transport	TCP (encrypted tunnel)	TCP
Network	Mesh overlay (WireGuard)	Ethernet (point-to-point)
Physical	Internet (Wi-Fi/LTE)	Cat5e cable

Security Model:

- 1. No public ports** — The mesh network uses NAT traversal; no firewall ports need to be opened
- 2. End-to-end encryption** — All traffic between nodes is encrypted with WireGuard (ChaCha20-Poly1305)
- 3. Identity-based access** — Each node has a cryptographic identity; only authorized nodes can join the mesh

4. **Firewall restricted** — Local firewall rules on the edge gateway only accept connections from mesh network IPs
5. **PLC program authority** — The PLC's ladder logic always has final authority over physical outputs, preventing remote override of safety systems

3. The Edge Gateway Module

The core innovation is a lightweight Python module (~100 lines) that provides three functions for remote PLC interaction:

`connect_to_edge(ip)` — Tests TCP connectivity to the PLC through the mesh network. Measures round-trip latency. Returns connection status.

`read_plc_registers(register_start, count, device_id, ip)` — Reads Modbus holding registers from the PLC. Returns raw register values and read latency. Used for monitoring analog values (motor speed, temperature, pressure, etc.).

`health_check(ip)` — Performs a full connectivity test plus a trial register read. Returns a structured health report with connectivity latency, Modbus protocol test result, and overall status (healthy/degraded/unhealthy).

The module uses the `pymodbus` library (v3.11+) for Modbus TCP communication with a 3-second timeout. Each function creates a fresh TCP connection, reads data, and closes — no persistent connections that could fail silently.

4. Modbus Address Map

The Allen-Bradley Micro 820 exposes its I/O through standard Modbus registers:

Coils (Boolean Values)

Address	Type	Description
0–6	Program Variables	motor_running, motor_stopped, fault_alarm, conveyor_running, sensor_1, sensor_2, e_stop
7–14	Digital Inputs	Physical switch positions, e-stop contacts, pushbuttons
15–17	Digital Outputs	Indicator LEDs, auxiliary relay

Holding Registers (16-bit Integer Values)

Address	Name	Scale	Unit
100	motor_speed	1.0	RPM
101	motor_current	0.01	Amps
102	temperature	0.1	°C
103	pressure	1.0	PSI
104	conveyor_speed	1.0	mm/s
105	error_code	1.0	—

Physical Control State Tables

3-Position Selector Switch:

Position	DI_00	DI_03	DO_00 (LED)
LEFT	0	0	0
CENTER	1	0	1
RIGHT	1	1	1

Emergency Stop:

State	DI_01 (NO)	DI_02 (NC)	DO_01 (LED)
Released (safe)	0	1	0
Pressed (stop)	1	0	1

5. Performance Results

Measured on February 18, 2026 with live hardware:

Metric	Value	Notes
Local Modbus read latency	3.85 ms	Edge gateway to PLC (direct Ethernet)
Mesh connectivity latency	28.88 ms	Cloud to edge gateway (encrypted tunnel)
End-to-end register read	33.28 ms	Cloud to PLC and back
End-to-end 6-register read	34.44 ms	Reading all process registers (100–105)
Health check total time	~62 ms	Connectivity test + trial read

Comparison to local HMI: A typical HMI panel refreshes at 100–500ms intervals. Our remote read latency of 33ms is **3–15x faster** than a local HMI update cycle.

Reliability: The mesh network automatically reconnects after interruptions. The Modbus client uses a 3-second timeout — if a read fails, the system reports degraded status rather than hanging.

6. Security Analysis

Attack Surface Assessment:

Vector	Mitigation
Internet-facing ports	None. Mesh uses NAT traversal — no listening ports on public IP
Man-in-the-middle	WireGuard (ChaCha20-Poly1305) encryption on all tunnel traffic
Unauthorized mesh access	Cryptographic node identity. Only approved nodes can join
Lateral movement	Edge gateway firewall restricts Modbus forwarding to mesh IPs only
PLC safety override	Ladder logic has absolute authority over physical outputs
Denial of service	3-second timeout on all Modbus operations. Graceful degradation

Defense-in-depth layers: 1. Mesh network authentication (node must be authorized) 2. Encrypted tunnel (all traffic encrypted) 3. OS-level firewall (port 502 restricted to mesh subnet) 4. Port forwarding (only forwards to PLC's specific IP) 5. PLC program logic (physical outputs controlled by ladder, not remote writes) 6. Application-level timeouts (prevents resource exhaustion)

PART 2: TECHNICIAN'S FIELD MANUAL

7. What You Need

Hardware Checklist:

- ☐ Allen-Bradley Micro 820 PLC (or compatible Modbus TCP device)
- ☐ Laptop or Raspberry Pi (the "edge gateway")
- ☐ Ethernet cable (Cat5e or better)
- ☐ Wi-Fi or cellular connection on the edge gateway
- ☐ Cloud server or VPS (any provider)

Software Checklist:

- ☐ Python 3.10+ on all machines
- ☐ pymodbus library (version 3.11+)
- ☐ Mesh VPN client (e.g., Tailscale) on edge gateway and cloud server
- ☐ FastAPI + Uvicorn (for the HTTP tag server)

Network Requirements:

- Edge gateway needs two network connections:
- **Ethernet** to the PLC (192.168.1.x subnet)
- **Wi-Fi/LTE** to the internet (for mesh VPN)
- Cloud server needs internet access and mesh VPN client

8. Step-by-Step Setup

8.1 Connect the Ethernet Cable

Plug a Cat5e Ethernet cable directly from the edge gateway's Ethernet port to the PLC's Ethernet port. No switch or router needed — direct connection works.

8.2 Set the Static IP

The edge gateway's Ethernet adapter must be on the same subnet as the PLC.

Example: If PLC is at 192.168.1.100, set the edge gateway to 192.168.1.50.

Windows (run as administrator):

```
netsh interface ipv4 set address name="Ethernet" static 192.168.1.50 255.255.255.0
```

Linux / Raspberry Pi:

```
sudo ip addr add 192.168.1.50/24 dev eth0
```

Verify:

```
ping 192.168.1.100
```

You should get responses in <1ms.

8.3 Verify PLC Connection

Run this Python snippet to confirm Modbus communication:

```
from pymodbus.client import ModbusTcpClient

client = ModbusTcpClient('192.168.1.100', port=502, timeout=3)
print(f"Connected: {client.connect()}")

# Read 18 coils (all I/O)
result = client.read_coils(address=0, count=18)
print(f"Coils: {[int(b) for b in result.bits[:18]]}")

# Read 6 holding registers (process values)
regs = client.read_holding_registers(address=100, count=6)
print(f"Registers: {regs.registers}")

client.close()
```

Expected output:


```
Connected: True
Coils: [0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0]
Registers: [0, 0, 0, 0, 0, 0]
```

The coil values will depend on the current state of the PLC's physical switches.

8.4 Start the PLC Backend (HTTP Tag Server)

This step exposes the PLC data as a clean HTTP/JSON API:

```
cd services/plc-modbus
python -m uvicorn backend.main:app --host 0.0.0.0 --port 8001
```

Then connect to the PLC:

```
curl -X POST http://localhost:8001/api/plc/connect \
  -H "Content-Type: application/json" \
  -d '{"ip": "192.168.1.100", "port": 502}'
```

Response: {"success": true, "message": "Connected to 192.168.1.100:502"}

Now read all I/O:

```
curl http://localhost:8001/api/plc/io
```

Response: Full JSON with coils, inputs, outputs, and registers — all named and structured.

8.5 Set Up the Port Forward

To allow remote servers to reach the PLC through the edge gateway's mesh network IP:

Windows (run as administrator):

```
netsh interface portproxy add v4tov4 ^
  listenaddress=<MESH_IP> listenport=502 ^
  connectaddress=192.168.1.100 connectport=502
```

Replace `<MESH_IP>` with the edge gateway's mesh network IP address.

Verify:

```
netsh interface portproxy show all
```

Linux:

```
socat TCP-LISTEN:502,bind=<MESH_IP>,fork TCP:192.168.1.100:502
```

8.6 Open the Firewall

Only allow connections from the mesh network:

Windows (run as administrator):

```
netsh advfirewall firewall add rule ^  
  name="Modbus TCP (Mesh Only)" ^  
  dir=in action=allow protocol=TCP localport=502 ^  
  remoteip=100.0.0.0/8
```

Also open the HTTP API port:

```
netsh advfirewall firewall add rule ^  
  name="PLC API (Mesh Only)" ^  
  dir=in action=allow protocol=TCP localport=8001 ^  
  remoteip=100.0.0.0/8
```

Linux:

```
sudo iptables -A INPUT -p tcp --dport 502 -s 100.0.0.0/8 -j ACCEPT  
sudo iptables -A INPUT -p tcp --dport 502 -j DROP
```

8.7 Test from Remote

From any authorized machine on the mesh network:

Health check (direct Modbus):

```
from integrations.edge_gateway import health_check
result = health_check('<MESH_IP>')
print(result)
# {"status": "healthy", "connectivity": {"latency_ms": 28.88},
#  "modbus_test": {"status": "success", "latency_ms": 33.28}}
```

Full I/O read (HTTP API):

```
curl http://<MESH_IP>:8001/api/plc/io
```

9. Quick Reference Card

Common Commands:

Task	Command
Check PLC connection	<code>curl http://localhost:8001/api/plc/status</code>
Read all I/O	<code>curl http://localhost:8001/api/plc/io</code>
Connect to PLC	<code>curl -X POST http://localhost:8001/api/plc/connect -H "Content-Type: application/json" -d '{"ip":"192.168.1.100"}'</code>
Check port forward	<code>netsh interface portproxy show all</code>
Remove port forward	<code>netsh interface portproxy delete v4tov4 listenaddress=<IP> listenport=502</code>
Check firewall rules	<code>netsh advfirewall firewall show rule name=all dir=in</code>

Troubleshooting:

Symptom	Check	Fix
"Connection timed out" to PLC	Ethernet cable connected?	Plug in cable, verify link light
"Connection timed out" to PLC	Correct subnet?	Set static IP: ipconfig should show 192.168.1.x
Remote server can't reach edge	Mesh VPN running?	Start mesh VPN client, verify mesh IP
Remote server can't reach PLC	Port forward set up?	Check netsh interface portproxy show all
Remote server can't reach PLC	Firewall open?	Add firewall rule for mesh subnet
API returns 503	PLC connection lost	Reconnect: POST /api/plc/connect
Coil writes don't stick	PLC program overriding	Normal — ladder logic has authority over physical outputs

10. API Reference

Base URL: `http://<edge-gateway>:8001/api`

GET /plc/status

Returns PLC connection status.

```
{
  "connected": true,
  "ip": "192.168.1.100",
  "port": 502,
  "last_seen": "2026-02-18T19:22:08"
}
```

GET /plc/io

Returns all PLC I/O as named, structured JSON.

```

{
  "coils": {
    "motor_running": false,
    "motor_stopped": false,
    "fault_alarm": false,
    "conveyor_running": false,
    "sensor_1_active": false,
    "sensor_2_active": false,
    "e_stop_active": false
  },
  "inputs": {
    "DI_00": true,
    "DI_01": false,
    "DI_02": true,
    "DI_03": false,
    "DI_04": false,
    "DI_05": false,
    "DI_06": false,
    "DI_07": false
  },
  "outputs": {
    "DO_00": true,
    "DO_01": false,
    "DO_03": false
  },
  "registers": {
    "motor_speed": 0,
    "motor_current": 0,
    "temperature": 0,
    "pressure": 0,
    "conveyor_speed": 0,
    "error_code": 0
  },
  "timestamp": "2026-02-18T19:22:08.729000"
}

```

POST /plc/connect

Connect to a PLC.

Request:

```
{"ip": "192.168.1.100", "port": 502}
```

Response:

```
{"success": true, "message": "Connected to 192.168.1.100:502"}
```

POST /plc/write-coil

Write a boolean value to a PLC coil (program variables 0–6 and outputs 15–17 only).

Request:

```
{"address": 0, "value": true}
```

Response:

```
{"success": true, "address": 0, "value": true, "name": "motor_running"}
```

APPENDICES

Appendix A: Glossary

Term	Definition
PLC	Programmable Logic Controller — a ruggedized computer that controls industrial machines
Modbus TCP	An industrial communication protocol that runs over Ethernet. Uses port 502
Coil	A boolean (on/off) value in a PLC — represents a digital input, output, or internal flag

Term	Definition
Holding Register	A 16-bit integer value in a PLC — represents an analog measurement or setpoint
Mesh VPN	A peer-to-peer encrypted network where every node can talk to every other node directly
WireGuard	A modern, fast VPN protocol used by mesh networks. Uses ChaCha20 encryption
Edge Gateway	The device that sits between the PLC network and the internet — bridges the two worlds
Ladder Logic	The programming language used in PLCs — looks like electrical relay diagrams
DI / DO	Digital Input / Digital Output — physical connections on the PLC for switches, sensors, LEDs
NAT Traversal	A technique that allows direct connections between devices behind firewalls/routers
HMI	Human-Machine Interface — a touchscreen panel mounted on a machine for local control
SCADA	Supervisory Control and Data Acquisition — centralized monitoring of industrial processes

Appendix B: PLC Ladder Logic Authority

A critical safety property of this system: **the PLC's ladder logic always wins.**

When a remote write command sets an output coil (e.g., DO_01), the value is written to the PLC's memory. However, if the PLC's ladder logic program also writes to that coil on its next scan cycle (typically every 10–50ms), the ladder logic value overwrites the remote value.

This means: - Physical safety outputs (e-stop LED, indicators) cannot be permanently overridden remotely - The PLC program is the final authority on all physical outputs - Remote writes to program variables (coils 0–6) persist because ladder logic may not actively write to them

This is a feature, not a bug. It ensures that remote diagnostics cannot interfere with machine safety.

Appendix C: Extending to Other PLC Types

The Modbus TCP protocol is universal. This system works with any PLC that supports Modbus TCP, including:

Manufacturer	Models	Notes
Allen-Bradley	Micro 820/850, CompactLogix	Enable Modbus in CCW/Studio 5000
Siemens	S7-1200, S7-1500	Use MB_SERVER function block
Schneider Electric	M340, M580	Native Modbus TCP support
Omron	NX/NJ Series	Use built-in Modbus server
Mitsubishi	iQ-R, iQ-F	Use SLMP/Modbus module
Automation Direct	Productivity, Click	Native Modbus TCP

To adapt to a different PLC, change only the address map (Section 4) and connection parameters. The edge gateway module, port forwarding, and mesh network are PLC-agnostic.

FactoryLM — AI for the Factory Floor

This whitepaper documents a verified implementation tested on live industrial hardware on February 18, 2026.